Template 1 Experiments

Thomas Ceresa June 8, 2026

q1: `?y spouse Einstein` **q2:** `?y docAdvisor Heisenberg` **q3:** `?x spouse Marić`

q4: `Curie award ?y` **q5:** `?x award TuringAward`

Setup. GPT-4o-mini (Azure, `temperature=0`), $N = 5$, Precision/Recall/F1 mean ± std. Two variants: **T1a** (bound term directly in pattern) vs **T1b** (bound term in FILTER). Cardinality: *low* ($|\text{GT}| \leq 2$), *medium* ($|\text{GT}| = 17$), *high* ($|\text{GT}| = 77$).

Strategies & Prompts.

NL — Natural language question, single call:

```
By your knowledge, who were
Albert Einstein's spouses?
List as many as you know.
```

SPARQL T1a — Raw SPARQL query, single call:

```
By your knowledge, execute this
SPARQL query and return all results:
SELECT ?y WHERE {
  Albert_Einstein spouse ?y }
Be exhaustive.
```

SPARQL T1b — With FILTER clause:

```
SELECT ?y WHERE {
  ?x spouse ?y .
  FILTER(?x = Albert_Einstein) }
```

ValueScan — Iterative triple pattern (max 5 iterations):

```
Task: List values of ?y such that:
(Albert Einstein, spouse, ?y)
factually holds.
Return as many values as you know.
[Iterative: already retrieved: {...}
 List more if any remain.]
```

All strategies use a shared system prompt: `You are an RDF knowledge graph assistant. Using your knowledge, respond ONLY in valid JSON: {"values":["..."]}`

Preliminary Study ($N = 3$, queries q1/q4/q5). I ran an ablation over URI encodings and prompt encodings across all strategies. I discovered empirically: (i) "NL-style URIs" outperform prefixed/full URIs — the LLM reasons better on plain strings than on `dbr:` or full HTTP URIs; (ii) Writing the triple as `(s, p, ?y)`) helps ValueScan's performences and ValueScan outperforms TripleScan; (iii) and exhaustive instructions (*"list all"*, *"be exhaustive"*) **stress the LLM** in NL contexts, cause empty responses on high-cardinality queries, while a softer invitation (*"list as many as you know"*) yields stable, non-empty outputs. However, SPARQL and ValueScan should be strict and exaustivity.

Results.

Table 1: F1 mean ± std ($N = 5$). GT = ground truth size. Best F1 per query in **bold**.

Query	GT	NL		SPARQL T1a		SPARQL T1b		ValueScan	
		F1	Gen	F1	Gen	F1	Gen	F1	Gen
q1 — spouse (obj, low)	2	**1.000**$_{\pm.000}$	2	.733$_{\pm.149}$	1	.667$_{\pm.000}$	1	**1.000**$_{\pm.000}$	2
q2 — docAdvisor (obj, low)	1	.000$_{\pm.000}$	1	.000$_{\pm.000}$	1	.000$_{\pm.000}$	2	**.667**$_{\pm.000}$	2
q3 — spouse (subj, low)	1	**1.000**$_{\pm.000}$	1	**1.000**$_{\pm.000}$	1	**1.000**$_{\pm.000}$	1	**1.000**$_{\pm.000}$	1
q4 — award (obj, med)	17	**.367**$_{\pm.046}$	7	.334$_{\pm.113}$	5	.210$_{\pm.000}$	2	.342$_{\pm.085}$	6
q5 — award (subj, high)	77	**.456**$_{\pm.023}$	34	.245$_{\pm.039}$	14	.181$_{\pm.068}$	10	.432$_{\pm.044}$	31

Discussion.

ValueScan vs. NL. On medium and high cardinality, ValueScan achieves higher *precision* (q5: .819 vs .751) while NL achieves higher *recall* (q5: .327 vs .304) and F1 on those queries. ValueScan prevents token-limit failures but also makes the LLM more conservative per iteration. NL, not over-constrained, uses parametric knowledge without stress but can hallucinate.

T1b < T1a. A FILTER clause rather than directly in the triple pattern consistently reduces recall — the LLM retrieves fewer results when the constraint is implicit.

Factual error in q2. All strategies except ValueScan consistently return "Max Born" instead of "Arnold Sommerfeld" across all $N = 5$ runs. I reckon that ValueScan, by iterating and being more strict, force the LLM to think twice and find rarer facts that a single call.

High cardinality limits. On q5 (77 expected values), all strategies plateau at 30–35 generated values, regardless of strategy (even with ValueScan with MAX_ITER=10, same plateau). In the PT, I saw LLM's the most frequent examples, consistently missing less prominent winners (e.g., Adi Shamir, Alan Kay, Alan Perlis appear in every FN list). That fits with what Galois affirms about the LLM's tendency of returning before popular, well-trained, data.